Assignment 7
Exception Handling in FP

Introduction to Functional Programming | Academic year 2026 2027

Maximum score 15 points | Individual work or teams of 2 to 4 students

Represent expected domain failures and missing values explicitly, and compose operations that may fail. Integrate Result, Maybe or Option, Map, Bind, and boundary exception handling into an existing project workflow.

Use your planned final project or a reusable module that will be integrated into it. Keep one runnable project across Assignments 5 to 8. A console interface is sufficient; a new application or a graphical interface is not required for each assignment.

Every team member must implement meaningful logic, contribute automated tests, and defend their own work. Agree on responsibilities before coding. Earlier relevant code and tests may be reused and improved.

Task 1 Define explicit domain outcomes

Implement or use a Result type with distinct success and failure cases. Success carries a value; failure carries a useful error code, message, or structured error. Protect the model against contradictory states, such as a success with a missing required payload or a failure treated as if its value exists.

Apply Result to at least three different expected domain failures. Examples are an invalid quantity, a missing required item, and an operation rejected because the current state does not permit it. Use failures meaningful to your application. Ordinary business failures must not depend on throwing and catching exceptions.

Define the expected successful result and the error for each failing case in README.md. Keep errors useful for the caller: identify the rule or item involved without silently returning an empty successful result.

Task 2 Model missing values

Use Maybe, Option, or an explicit Found and NotFound model for at least one real lookup. Handle both present and absent cases. For a primitive value, zero, false, or an empty string may be legitimate data; do not automatically equate them with absence.

If the workflow requires the item, convert the missing case into a meaningful Result failure. If the item is optional, apply and document the intended optional behaviour. Do not create a fabricated record, rely on default(T) as proof of absence, or use the null-forgiving operator to skip the case.

You may use your Assignment 6 case model and factory outcomes where appropriate. A validation failure and a workflow state have different meanings; explain what each type represents.

Composition and boundary handling

Task 3 Implement and use Map and Bind

Use Map to transform a successful value with a pure function. Use Bind to continue with an operation that itself returns Result. Build one real project chain that invokes both. The following signatures illustrate their roles; adapt the error type and method names to your model.

```
Map  : Result<T, E> + Func<T, U>          -> Result<U, E>
Bind : Result<T, E> + Func<T, Result<U,E>> -> Result<U, E>
```

On failure, both operations must propagate the failure without calling the supplied function. Do not access the success payload until the success case has been established. Handle the final outcome once at the application boundary, using pattern matching or another explicit case handler.

For example, a quote workflow can find an item, validate the request, calculate a price, and create a quote. A missing item stops validation and calculation. Map can create the final display model from a successful quote; Bind can continue from validation to an operation that may reject the request. The actual business rules remain your choice.

Task 4 Handle operational exceptions at boundaries

Load project data from JSON or CSV. Handle a missing data file and malformed content. Catch the relevant file or parsing exception at the boundary and convert it into a clear outcome. Keep file access outside pure domain functions.

Distinguish expected domain failures from operational problems. Report enough context to identify the operation or file. An empty catch, a catch-all handler that claims success, or a fallback that silently discards the problem is unacceptable. A failure must not leave partially updated domain or stored data.

If your program intentionally supports a fallback, state when it applies and make the degraded outcome visible. The missing-file and malformed-content demonstrations must still show that the problem was detected.

Task 5 Integrate the outcomes into the application

Provide a normal path, the three domain failure paths, present and missing lookup paths, and the two file failure paths. Keep the earlier pipeline and safe state transitions active. The user must receive a clear result or explanation of the failure without an unhandled exception for these demonstrated cases.

Optional practice with recursion

If the project naturally contains nested categories, dependencies, or a tree, you may add a recursive operation with a clear base case and progress toward it. Test empty and nested structures. Trampolining may be explored after its lecture if deep recursion is relevant. These topics are optional and do not replace the required error handling or earn extra rubric points.

Tests and individual defense

Provide at least 10 cumulative automated tests. Use an assertions-based test framework or a test runner that compares actual and expected values and reports failures. Console output alone is not a test. Relevant earlier tests count toward the minimum.

The cases below define required coverage. One test may contain several assertions, and one scenario may cover several requirements. Use additional tests if needed; the minimum number does not remove any required case.

CASE	REQUIRED ASSERTION
COMPLETE SUCCESS	The integrated Result chain produces the correct value.
DOMAIN FAILURES	Check each of at least three distinct domain failures and its useful error.
LOOKUP PRESENT AND ABSENT	Both cases work; legitimate primitive values are not mistaken for absence.
MAP SUCCESS	The transformation receives the successful value and returns the expected new value.
BIND SUCCESS	The next operation receives the successful value and its Result is preserved.
FAILURE PROPAGATION	Map and Bind preserve a failure and do not invoke their supplied functions. Use a counter, flag, or test double to assert non-execution.
MISSING FILE	The boundary reports the problem and preserves consistent state.
MALFORMED FILE	Invalid JSON or CSV becomes a clear failure without a partial update.

Keep test data under your test project or an explicit sample-data folder. File tests must use controlled temporary paths and clean up resources; do not depend on a file that happens to exist on your laptop.

What to show during the defense

- Run a successful workflow, one domain failure, a missing lookup, and a file failure.
- Trace the Result chain and explain where Map and Bind are used and why their return types differ.
- Show the assertion that proves a dependent operation is skipped after failure.
- Explain how valid zero or false differs from an absent value when relevant to your model.
- Identify the exception boundary and explain how domain state is preserved after failure.
- Add or modify a domain rule or an error-handling test and verify the result.

Assessment and submission

Assessment

CRITERION	EVIDENCE ASSESSED	POINTS
DOMAIN OUTCOMES	At least three useful explicit domain failures and correct success behaviour.	1
RESULT COMPOSITION	Used Map and Bind with correct transformation, continuation, and failure propagation.	2
BOUNDARY HANDLING	Missing files and malformed content produce clear outcomes without partial updates.	2
MISSING VALUES	A real lookup handles present and absent cases explicitly, including relevant primitive values.	1
TESTS AND DOCUMENTATION	At least 10 cumulative tests, proof of skipped dependent functions, and documented outcomes.	2
INDIVIDUAL DEFENSE	Explain your error handling and complete a small live modification.	7
TOTAL		**15**

The implementation criteria total 8 points; the individual defense is worth 7 points. Each student receives an individual grade. Only implemented constructs used by the application earn points. A student who does not defend receives 0. If you cannot explain your solution, it is not accepted under the course code defense policy. Academic integrity and AI rules follow the syllabus.

What to upload

Every student must upload a ZIP archive with a runnable snapshot for Assignment 7. Team members may upload the same project code, but each must identify and defend their own contribution. Use the deadline and submission policy published in Moodle.

- Include C# source, project and solution files, automated tests, README.md, and sample input files needed to run the demonstrated workflows. Exclude generated .NET bin and obj folders, IDE caches, and local secrets.
- State the .NET SDK version, exact build, run and test commands, expected working directory, and any prerequisites. Check these instructions from a clean extraction of the archive.

README content

- Describe the project workflow, its input, domain rules, expected output, and short instructions for the normal and failure demonstrations.
- Map each requirement to the files and methods that implement it. List test cases, expected results, and the latest test outcome.
- List each member by name and group and explain their implemented logic, tests, and documentation for this stage. Team size does not increase the number of required workflows or reports.

Use in the final project

Retain the code and tests for the next stage. The final project must be hosted in an accessible Git repository on GitHub, GitLab, or another Git hosting service. Repository links do not replace the files required in Moodle. The final project has a separate assessment of 50 points for the project and 50 points for the individual defense, with a group project report and an individual contribution report.